Szoftvertechnológia (VIMIAB04)

Teszttervezési technikák

Elekes Márton, Dr. Majzik István, Dr. Micskei Zoltán
https://ftsrg.mit.bme.hu

MŰEGYETEM 1782

BME VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR

Mesterséges Intelligencia és Rendszertervezés Tanszék

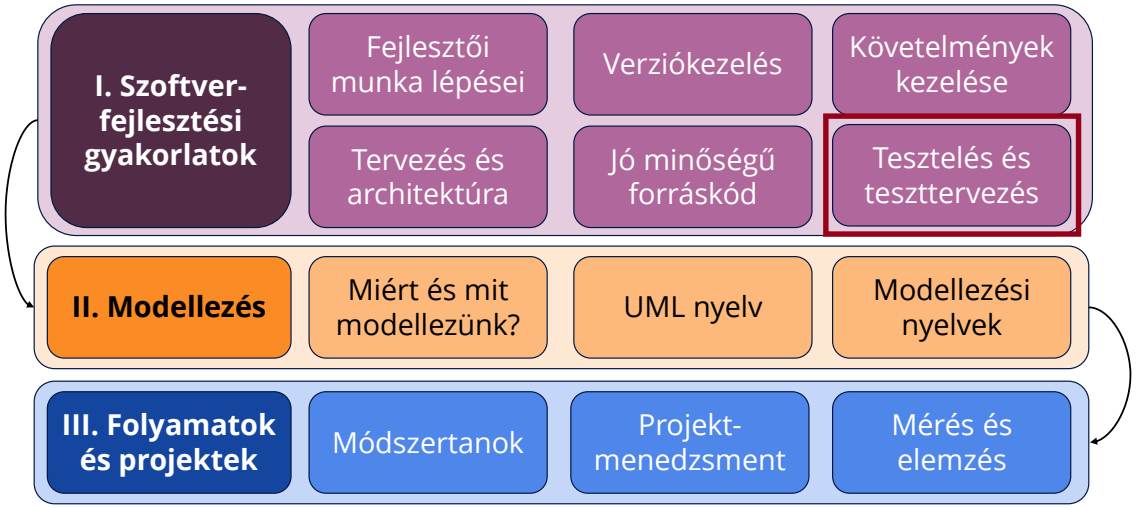
ftsrg Critical Systems Research Group

Tanulási eredmények

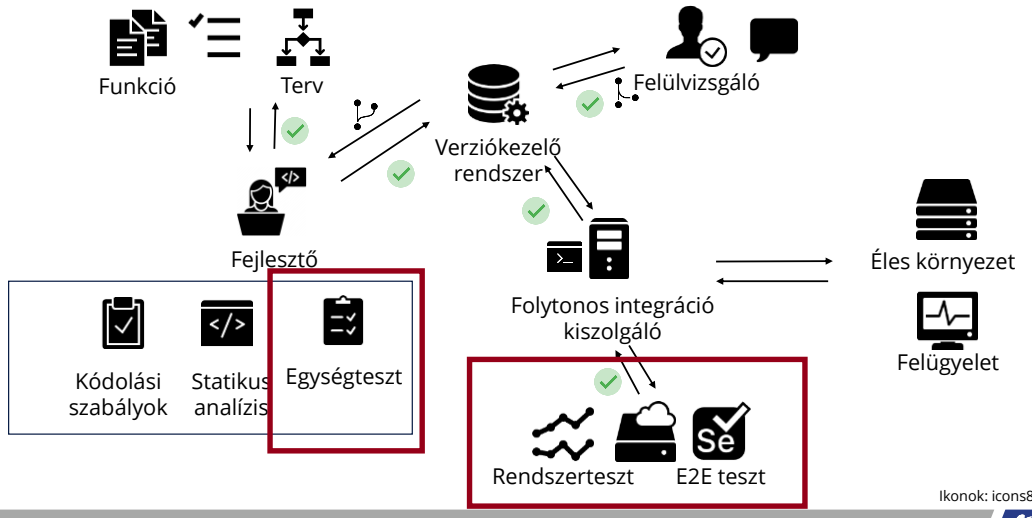
A tananyag elsajátítása után a hallgató képes lesz

- (K2) összefoglalni a teszttervezés kihívásait,
- (K3) alkalmazni alapvető specifikációalapú teszttervezési technikákat,
- (K3) alkalmazni alapvető struktúraalapú teszttervezési technikákat.

Szoftvertechnológia tantárgy témái



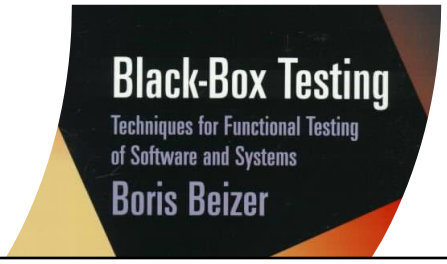
Tipikus fejlesztési munkafolyamat



Ikonok: icons8.com

Miért fontos a teszttervezés?

Nemcsak a tesztelés, hanem még inkább a tesztek tervezése az egyik legjobb a hiba megelőzésében.

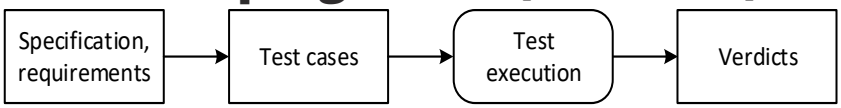


5

„More than the act of testing, the act of designing tests is one of the best bug preventers known.”

Sokszor nem is a tesztek lefuttatása és az eredmények megfigyelése, hanem már a tesztek megtervezése során is felfedezünk ellentmondásokat, hiányosságokat, kérdéseket, amik növelik a termék/rendszer minőségét. Nagyon hasznos, ha végig gondoljuk, hogy az egyes funkciók működését hogyan tudjuk ellenőrizni, milyen hibaesetekre kell felkészülni, hogyan tudjuk a rendszert a tesztelés során vezérelni és megfigyelni.

Tesztelési alapfogalmak (ismétlés)



- SUT: system under test
- Teszteset (test case)
  - bemeneti értékek, végrehajtási feltételek és elvárt eredmények halmaza, amelyeket egy konkrét célért fejlesztettek
- Tesztkészlet (test suite)
- Teszt-orákulum (test oracle)
  - Olyan elv vagy módszer, ami segít eldönteni a tesztelőnek, hogy sikeres volt-e a teszt
- Eredmény (verdict): pass / fail / error / inconclusive…

Teszttervezési technikák

Cél: tesztesetek kiválasztása tesztcél alapján

Specifikáció alapján

- SUT: black box
- Csak a specifikáció ismert
- Előírt funkcionalitás vizsgálata

Struktúra alapján

- SUT: white box
- Belső felépítés ismert
- Felépítés és működés alapján bejárás

Lefedettségi metrikák

- Tesztelhető elemek mekkora %-a volt tesztelve

- Tesztelhető elemek
  - Specifikáció: követelmény, funkció…
  - Struktúra: utasítás, döntés…

- Lefedettségi feltétel (coverage criterion)
  - *X* % az *Y* metrikánál

- Ez nem hibafedés!

Hibafedés: Mivel nem ismerjük, hogy ténylegesen mennyi hiba van a tesztelt rendszerben, ezért sosem tudhatjuk biztosan mondani, hogy a hibák X%-át megtaláltuk. (Ha ismernénk az összes hibát, akkor nem kéne tesztelni.) Helyette ilyen áttételes metrikákat fogunk tudni használni.

Mire jók a lefedettségi metrikák?

Kiértékelés (mérés)	Kiválasztás (cél)
• Meglévő tesztek kiértékelése • Hiányzó tesztek azonosítása	• Teszt tervezése metrika mentén • Cél adott % elérése

- Pl. egy követelmények alapján készített tesztkészletnél megmérjük, hogy a forráskód utasításainak hány százalékát hajtjuk végre legalább egyszer a tesztek futtatása során. Ha hiányzik valami, akkor az azonosított hiányosságok alapján tudunk új teszteket tervezni.
- Vannak olyan algoritmusok és tesztgeneráló eszközök, amik adott metrika alapján tudnak tesztbemeneteket generálni, például az összes kódban lévő elágazás lefedését megcélozva.

Specifikáció alapú tesztelés

- Ekvivalencia-particionálás
- Határérték-analízis
- Use case / user story
- Kombinatorikus módszerek
- Döntési táblák
- ...

Teszttervezési technikák

Cél: tesztesetek kiválasztása tesztcél alapján

Specifikáció alapján

- SUT: black box
- Csak a specifikáció ismert
- Előírt funkcionalitás vizsgálata

Struktúra alapján

- SUT: white box
- Belső felépítés ismert
- Felépítés és működés alapján bejárás

Specifikáció alapú technikák

- Ekvivalencia-particionálás
- Határérték-analízis
- Use case / user story
- Kombinatorikus módszerek
- Döntési táblák
- ...

- A tantárgyban az első három metrikát tárgyaljuk most, a többi MSc tantárgyakban vagy például az ISTQB Advanced tananyagában előkerül.
- A Kombinatorikus módszerek (pl. pair-wise testing) sok bemeneti paraméter esetén tudnak egy hibafedés szempontjából hatékony részhalmazt kiválasztani.
- A döntési táblák (decision table) a bemenetek és kimenet közötti (logikai) kapcsolatokat foglalják össze átlátható formában, és segítik a teszttervezést.

Ekvivalenciaparticionálás

- Bemenet és kimenet ekvivalenciaosztályai:
  - Olyan adatok, amelyek várhatóan ugyanazt a hibát fedik le (ugyanazt a programrészt járják be)
  - Cél: Egy-egy ekvivalencia osztályból egy-egy teszt adat kiválasztása (többire elvileg hasonló)
- Nagyon környezetfüggő (context-dependent)
  - Ismerni kell a környezetet és a SUT-ot!
  - Tesztelő tudásán múlik a módszer hatékonysága

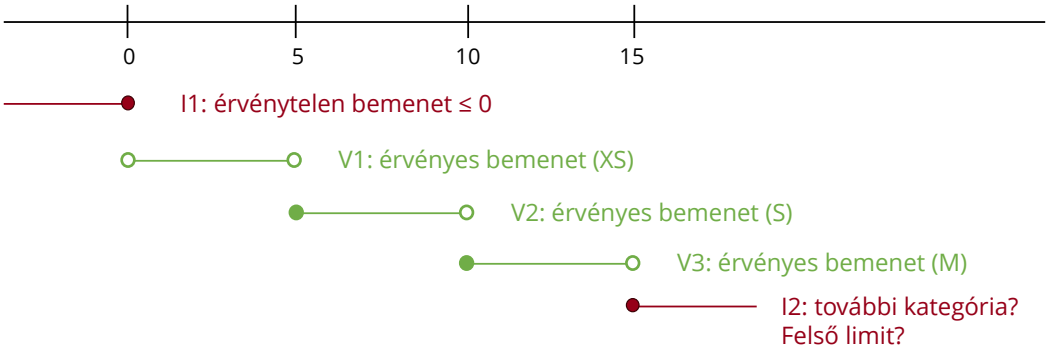
- Angolul: equivalence partitioning
- Ekvivalenciaosztály: „Egy komponensben vagy rendszerben található változó értékkészletének egy részhalmaza, melyben a specifikáció alapján minden érték azonosan kezelendő.” [ISTQB]
- Ekvivalenciaparticionálás: „Feketedoboz-teszttechnika, melyben a tesztfeltételek ekvivalenciapartíciók, és mindegyik partíciót annak egy reprezentatív elemével fedünk le.” [ISTQB]

Ekvivalenciaosztályok meghatározása

- Meghatározás heurisztikus folyamat:
  - Kezdés: érvényes és érvénytelen bemeneti adatok
  - Partíciók tovább finomítása
- Tipikus heurisztikák:
  - Tartomány (pl. 1-1000)
    - < min, min-max, >max
  - Halmaz (pl. RED, GREEN, BLUE)
    - érvényes elem, érvénytelen
  - Specifikus (pl. @ az elején)
    - feltétel teljesül, feltétel nem teljesül
  - Egyéni kivételek (pl. február hónap)

Példa: ekvivalenciaosztályok

„A csomagfeladás felületén megadjuk a csomag súlyát, és a rendszer kiírja, hogy az XS-es (<5kg), S-es (<10kg) vagy M-es (<15).”



Egy-egy partíción belül az értékek remélhetőleg hasonlóan viselkednek (pl. 3 és 4 kg esetén ugyanúgy vagy helyes a működés vagy hibás). Határértékekről később lesz szó.

Tesztesetek származtatása ekv. osztályból

- Több bemenet ekv. osztályainak összekapcsolása
- Érvényes (normál) ekv. osztályok esetén:
  - egy teszt minél több osztályt fedjen le
- Érvénytelen ekv. osztályok esetén:
  - először minden érvénytelen osztályhoz külön teszt legyen
    - egymás hatását ne oltsák ki
  - majd több osztály kombinációja is

FELADAT: Ekvivalenciaparticionálás

Követelmény: Új akciót kell implementálni a webes felületen. Ha megadja a vevő a törzsvásárlói számát, akkor legalább 2 hamburger rendelése esetén kap egyet ingyen. Ha menüben (burger, krumpli, üdítő) rendeli a hamburgereket, akkor az üdítőért nem kell fizetni.

- Bemeneti paraméterek? Ekvivalencia osztályok?
- Van kérdésünk a követelménnyel kapcsolatban?

Specifikáció alapú technikák

- Ekvivalencia-particionálás
- Határérték-analízis
- Use case / user story
- Kombinatorikus módszerek
- Döntési táblák
- ...

Határérték-analízis

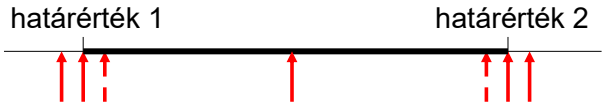
- Adattartományok határait vizsgálja
  - Egy-egy ekvivalencia osztály határaira koncentrál
  - Bemeneti és kimeneti tartományokra is
- Tipikus megtalált hibák:
  - relációs operátorok
  - ciklusok be- és kilépési feltételei
  - adatstruktúrák mérete
  - …

- Angolul: Boundary Value Analysis (BVA)
- Határérték: „Egy rendezett ekvivalenciapartíció minimum vagy maximum értéke.” [ISTQB]
- Határérték-elemzés: „Feketedoboz-teszttechnika, melyben a tesztesetek tervezése a határértékek alapján történik.” [ISTQB]

Tipikus teszt adatok határértékeknél

- Egy határérték 3 tesztet jelent:

- Egy tartomány 5-7 tesztet jelent:



- Egész értékkészletű paraméter esetén könnyű a határértéknél eggyel kisebb/nagyobb értéket értelmezni.
- Valós szám esetén ez már egy sokkal nehezebb kérdés, mert eleve a határértéket se biztos, hogy ki tudjuk választani. Fontos ilyenkor, hogy milyen pontossággal kell dolgozni.

FELADAT: Határérték-analízis

Követelmény: Ha a sikertelen kérések száma eléri az ötöt, akkor email értesítést kell küldeni az operátornak. Ha a sikertelen kérések száma meghaladja a tízet, akkor SMS-riasztást kell küldeni.

- Milyen értékeket használjunk a tesztelésre?
- Van kérdésünk a követelménnyel kapcsolatban?

Specifikáció alapú technikák

- Ekvivalencia-particionálás
- Határérték-analízis
- Use case / user story
- Kombinatorikus módszerek
- Döntési táblák
- ...

Tesztek származtatása használati esetekből

- Használati eset (use case): egy fő funkció forgatókönyve (lásd UML)

- Tipikus tesztesetek:
  - 1 teszt: fő ág („happy path”, „mainstream”)
    - Ellenőrzés: utófeltételek vizsgálata
  - Alternatív lefutások: mindegyikhez külön teszteset
  - Előfeltételek (nem)teljesülése

- Jobbára magasabb szintek (rendszer, elfogadási teszt, …)

- A fő ág (happy path) tesztelésével érdemes kezdeni, hiszen ha az nem működik, akkor nem is biztos, hogy érdemes folytatni a tesztelést (ha nem tud egy „egyszerű” adatokkal rendelkező felhasználó regisztrálni, akkor már mindegy, hogy kezeli-e a rendszer az idegen írásmódú neveket vagy sem).
- Előfeltételek nem teljesülése esetén az az elvárás, hogy a rendszer kezeli a hibás helyzetet, és például megfelelő hibaüzenetet ad.

FELADAT: Használati esetek (use case)

3.2.5 Vásárlás

ID / Név:	UC6 / Buy
Verzió:	1.0
Leírás:	A felhasználó a megvásárolni kívánt könyvek kosárba tétele után kifizetheti azokat, ha megad ehhez egy érvényes bankkártya számot, amiről a vételár levonható.
Előfeltétel:	Van legalább egy könyv a felhasználó kosarában, megadott egy érvényes bankkártya számot a kosár megtekintésénél és ezt követően nem navigált el a kosár tartalmát listázó oldalról.
Utófeltétel:	Az ügyfél kosara kiürül, és a könyveket megvásárolja.
Trigger:	A felhasználó a fizetés funkciót választja.
Normál lefutás:	1. A kosárban lévő könyv példányok kikerülnek az adatbázisból. 2. A kosár is kiürül. 3. A fizetés ténye belekerül a tranzakció naplóba.
Alternatív lefutások:	- Ha nincs megadva vagy érvénytelen a bankkártya szám, akkor nem változik sem a készleten lévő, sem a kosárban lévő könyvek listája.

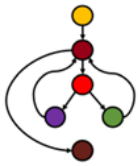
- Ezek már nem egységtesztek, a teszteseteknek nem csak a konkrét művelet és paramétere lesz a része, hanem a (teszt)rendszer állapota (milyen tesztadatot rögzítettünk korábban, milyen könyvek vannak, milyen felhasználók vannak az adatbázisban).

Struktúra alapú tesztelés

Forráskód:

```
int a = read();
while(a < 16) {
  if(a < 10) {
      a += 2;
  } else {
      a++;
  }
}
a = a * 2;
```

Control-flow graph (CFG):



Teszttervezési technikák

Cél: tesztesetek kiválasztása tesztcél alapján

Specifikáció alapján

- SUT: black box
- Csak a specifikáció ismert
- Előírt funkcionalitás vizsgálata

Struktúra alapján

- SUT: white box
- Belső felépítés ismert
- Felépítés és működés alapján bejárás

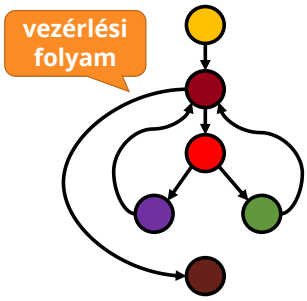
Mi a belső felépítés?

Kód esetén: forráskód struktúrája

Forráskód:

```
int a = read();
while(a < 16) {
  if(a < 10) {
      a += 2;
  } else {
      a++;
  }
}
a = a * 2;
```

Vezérlésifolyam-gráf (CFG):



Megjegyzés: CFG pontos definíciójával most nem foglalkozunk

- Control-flow graph (CFG): vezérlésifolyam-gráf
- A különböző megoldások és definíciók eltérnek abban, hogy például a feltétel és ciklus utasításoknak felvesznek-e külön csomópontot a gráfban. Például az if-et jelző piros csomópont kiegyszerűsíthető könnyen a gráfból, és nem változik a CFG struktúrája.
- Az egyszerűség kedvéért érdemes most kezdésnek úgy rajzolni majd CFG-t, hogy minden utasításhoz felveszünk egy csomópontot, nem csak ahhoz, aminek „pontosvessző van a végén”. Külön érdemes figyelni a for ciklusokra, mert az több utasításból áll!

Alapfogalmak

```
int t = 1;
Speed s = SLOW;

if (! started){
  start();
}

if (t > 10 && s == FAST){
  brake();
} else {
  accelerate();
}
```

Utasítás

Blokk

Feltétel

Döntés

Döntési ág

Alapfogalmak

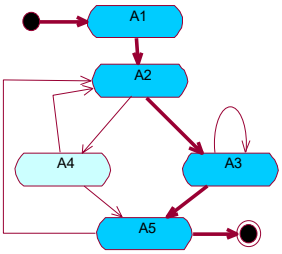
- Utasítás (instruction)
- Blokk (block)
  - Utasítások egybefüggő sorozata, amik között nincs elágazás vagy függvényhívás
- Feltétel (condition)
  - Egyszerű vizsgálat, amiben nincs logikai (Boole) operátor
- Döntés (decision)
  - Nulla vagy több logikai operátorral összekötött feltételből álló kifejezés
- Döntési ág (branch)
  - Egy döntés lehetséges kimenetele
- Út (path)
  - Utasítások sorozata, tipikusan a modul be és kilépési pontja között

Sajnos ezeknek az alapvető fogalmaknak a definíciójában nincs teljes egyetértés (pl. ISTQB vs. DO-178B vs. IEEE).

Döntés:
- (ISTQB): A program point at which the control flow can divide into various paths.
- DO178B-ben ez viszont nincs benne, így ott a következő utasítás is döntésnek minősül, holott nincs benne branching
  - A = B or C
- Most a tantárgyban mi is egyelőre azt vesszük döntésnek, ahol elágazik a program vezérlése (feltétel, ciklus, kivétel…). A döntés áll(hat) feltételekből, és van legalább kettő döntési ága (amerre mehet tovább a vezérlés).

1. Utasítás lefedettség

$$\frac{\text{Tesztelés során végrehajtott utasítások száma}}{\text{Összes utasítás szám}}$$



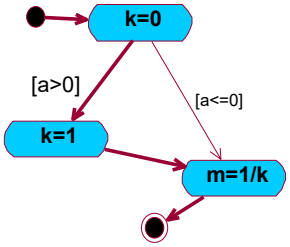
Utasítás lefedettség: 4/5 = 80%

- Előkerül még a line coverage elnevezés is, ami nem szerencsés, mert egy sorban lehet több utasítás is.

Utasítás lefedettség értékelése

Minden utasítást legalább egyszer végrehajtunk

```
k = 0;
if (a > 0){
  k = 1;
} // no else branch
m = 1 / k;
```



Utasítás lefedettség: 100%

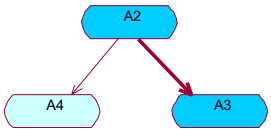
DE: hiányzik a [a<=0] ág

Üres ágak lefedését nem garantálja


```
k = 0;
if (a > 0){ k = 1 }
// no else branch
m = 1 / k;
```

2. Döntési lefedettség

$$\frac{\text{Tesztelés során előfordult döntési eredmények száma}}{\text{Döntési eredmények lehetséges száma}}$$



Döntési lefedettség: 50%

Hány eredménye lehet egy döntésnek?

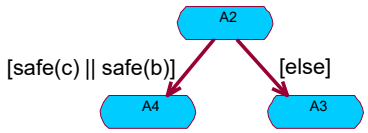
Nem kell a döntések kombinációját is figyelni, csak külön-külön!

- Ebben az esetben két eredménye van a döntésnek, de a switch utasítás esetén lehet több is.
- Használják még a branch coverage elnevezést is, azonban az például repülős környezetben nem ugyanaz mint a decision coverage, lásd:
FAA, Certification Authorities Software Team, „What is a "Decision" in Application of Modified Condition/Decision Coverage (MC/DC) and Decision Coverage (DC)?”, 2002. URL:
http://www.faa.gov/aircraft/air_cert/design_approvals/air_software/cast/cast_papers/media/cast-10.pdf

Döntési lefedettség értékelése

Minden utasítást legalább egyszer végrehajtunk

Minden ágat lefedünk (üres ágakat is)



#	safe(c)	safe(b)	safe(c) || safe(b)
1	T	F	T
2	F	F	F

100% döntési lefedettség

DE: safe(b) = true eset hiányzik!

Feltételek kombinációjára nem figyel

További lefedettségi metrikák (lásd MSc)

- Condition Coverage
- Condition/Decision Coverage (C/DC)
- Modified Condition/Decision Coverage (MC/DC)
- Multiple Condition Coverage (MCC)
- Loop Coverage
- …
- All-Defs Coverage
- All-Uses Coverage
- …

FELADAT: Struktúra alapú tesztelés

```
1 int pow(int n, int k) {
2   if (n < 0 || k < 0) {
3     return -1;
4   }
5   int p = 1;
6   for (int i = 0; i < k; i++) {
7     p *= n;
8   }
9   return p;
  }
```

Rajzold fel a kód CFG-jét!
Tervezz teszteket:

- 100% utasítás lefedettség
- 100% döntési lefedettség

Kód lefedettség számolása a gyakorlatban

- Mindegyik eszköznél más definíciók
- Megvalósítás
  - Forrás/bináris kód instrumentálása
  - Lefedettséget számoló utasítások hozzáadása

```
if (a > 10){
  CoveredBranch(1, true);
  b = 3;
} else {
  CoveredBranch(1, false);
  b = 5;
}
send(b);
```

Lásd például: Code Coverage Differences of Java Bytecode and Source Code Instrumentation Tools

Kód lefedettség használata

Mire használható

- Egyáltalán nem tesztelt kódrészlet azonosítása
- Tesztkészlet „teljességének” értékelése
- Kilépési feltételekben

Mire nem használható

- Hiányzó/hiányos követelmények azonosítása
- Kód/teszt minőségével csak indirekt kapcsolat

Kód lefedettség használata a gyakorlatban

- Microsoft tapasztalata:
  - „Test suite with high code coverage and high assertion density is a good indicator for code quality.”
  - „Code coverage alone is generally not enough to ensure a good quality of unit tests and should be used with care.”
  - „The lack of code coverage to the contrary clearly indicates a risk, as many behaviors are untested.”

    (Forrás: „Parameterized Unit Testing with Microsoft Pex”)
- Kapcsolódó cikkek:
  - „*Coverage Is Not Strongly Correlated with Test Suite Effectiveness*”, 2014. DOI: 10.1145/2568225.2568271
  - „*The Risks of Coverage-Directed Test Case Generation*”, 2015. DOI: 10.1109/TSE.2015.2421011

Teszttervezési technikák összefoglalása

- Specifikáció és struktúra alapú technikák
  - Több, egymást kiegészítő módszer
  - Mindegyik módszert gyakorolni kell
- Technikák kombinációja hasznos
  - Példa (*How We Test Software at Microsoft*):
    specifikáció alapú: 83% kód lefedettség
    + felderítő tesztelés: 86% kód lefedettség
    + struktúra alapú: 91% kód lefedettség

Összefoglalás

Teszttervezési technikák

Cél: tesztesetek kiválasztása tesztcél alapján

Specifikáció alapján

- SUT: black box
- Csak a specifikáció ismert
- Előírt funkcionalitás vizsgálata

Struktúra alapján

- SUT: white box
- Belső felépítés ismert
- Felépítés és működés alapján bejárás

Specifikáció alapú technikák

Ekvivalencia partícionálás

Határérték-analízis

Use case / user story

Kombinatorikus módszerek

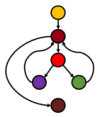
Döntési táblák

...

Mi a belső felépítés?

Kód esetén: forráskód struktúrája

Forráskód:

```
int a = read();
while(a < 16) {
  if(a < 10) {
      a += 2;
  } else {
      a++;
  }
}
a = a * 2;
```

Control-flow graph (CFG):